

Lab 5: FSM – Finite State Machine

Name: **Loc D. T. Nguyen**

Student ID: **24125093**

March 23, 2026

1 Exercise 1: Moore FSM – Sequence Detector “101”

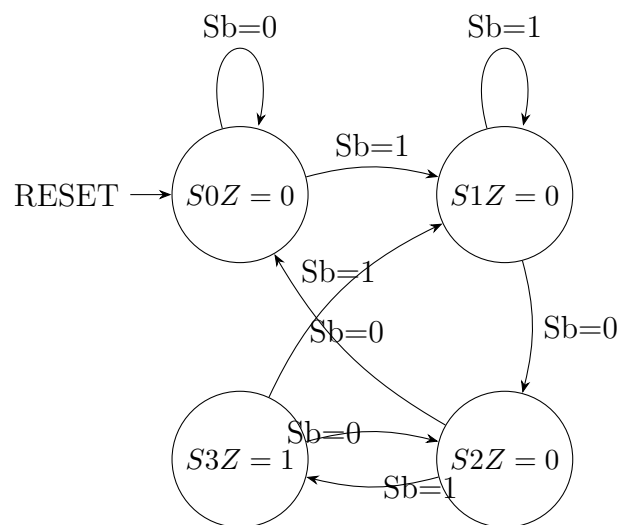
1.1 Overview

A Moore FSM is designed to detect the bit sequence “101” in a serial data stream, with support for overlapping sequences. In a Moore FSM, the output depends *only* on the current state (not on the current input). The output $Z = 1$ is asserted one clock cycle after the third bit of the pattern is received.

1.2 State Transition Diagram

The FSM requires four states to track progress through the pattern:

- **S0** ($Z = 0$): Reset / initial state – no useful bits received yet.
- **S1** ($Z = 0$): Received “1” (first bit of pattern).
- **S2** ($Z = 0$): Received “10” (first two bits of pattern).
- **S3** ($Z = 1$): Received “101” – pattern complete; output is asserted.



1.3 State Table

Current State	Next State (Sb=0)	Next State (Sb=1)	Output Z
S0	S0	S1	0
S1	S2	S1	0
S2	S0	S3	0
S3	S2	S1	1

1.4 State Encoding

Each state is assigned a 2-bit binary code Q_1Q_0 :

State	Q_1Q_0
S0	00
S1	01
S2	10
S3	11

1.5 Next-State (D Flip-Flop Input) Table

Using D flip-flops, the D inputs equal the desired next-state values directly:

Q_1Q_0	Next State Q_1Q_0		D Inputs D_1D_0	
	Sb=0	Sb=1	Sb=0	Sb=1
00	00	01	00	01
01	10	01	10	01
10	00	11	00	11
11	10	01	10	01

1.6 Output and Flip-Flop Input Logic

Output Function

The output Z depends only on the current state (Moore). It is 1 only in state S3 ($Q_1Q_0 = 11$):

$$Z = Q_1 \cdot Q_0$$

D_1 (Next Q_1)

$D_1 = 1$ when the next state has $Q_1 = 1$, i.e. next states S2 (10) or S3 (11). From the transition table:

- $S1 \rightarrow S2$ when $Sb = 0$: contributes $\overline{Q_1} \cdot Q_0 \cdot \overline{Sb}$
- $S2 \rightarrow S3$ when $Sb = 1$: contributes $Q_1 \cdot \overline{Q_0} \cdot Sb$
- $S3 \rightarrow S2$ when $Sb = 0$: contributes $Q_1 \cdot Q_0 \cdot \overline{Sb}$

Combining and simplifying:

$$D_1 = Q_1 \cdot \overline{Sb} + \overline{Q_1} \cdot Q_0 \cdot \overline{Sb}$$

which simplifies to:

$$D_1 = \overline{Q_0} \oplus \overline{Q_1} \cdot \overline{Sb} + \overline{Q_1} \cdot Q_0 \cdot \overline{Sb} = Q_0 \cdot \overline{Sb} \cdot \overline{Q_1} + Q_1 \cdot \overline{Q_0} \cdot Sb$$

More directly from the D-table:

$$D_1 = \overline{Q_1} \cdot Q_0 \cdot \overline{Sb} + Q_1 \cdot \overline{Q_0} \cdot Sb + Q_1 \cdot Q_0 \cdot \overline{Sb}$$

D_0 (**Next** Q_0)

$D_0 = 1$ when the next state has $Q_0 = 1$, i.e. next states S1 (01) or S3 (11). This occurs whenever $S_b = 1$ (for any current state in this FSM):

$$D_0 = S_b$$

Verification with Example

Given $A = 1011010100101$, the expected output is $Z = 0010010100001$:

Bit #	Input S_b	Current State	Next State	Output Z
–	–	S0	–	0
1	1	S0	S1	0
2	0	S1	S2	0
3	1	S2	S3	0
4	1	S3	S1	1
5	0	S1	S2	0
6	1	S2	S3	0
7	0	S3	S2	1
8	1	S2	S3	0
9	0	S3	S2	1
10	0	S2	S0	0
11	1	S0	S1	0
12	0	S1	S2	0
13	1	S2	S3	0

The output Z (read from the “Current State” column, one cycle delayed) matches the expected pattern, confirming correctness.

1.7 Logisim Implementation

The Moore FSM is implemented in Logisim as follows:

1. Two **D Flip-Flops** store the current state bits Q_1 and Q_0 .
2. **Input logic** computes D_1 and D_0 using AND/OR/NOT gates per the derived equations.
3. **Output logic** computes $Z = Q_1 \cdot Q_0$ using a single AND gate.
4. A **Reset** signal initialises both flip-flops to 0 (state S0).
5. A clock drives both flip-flops synchronously.

2 Exercise 2: Mealy FSM – Sequence Detector “101”

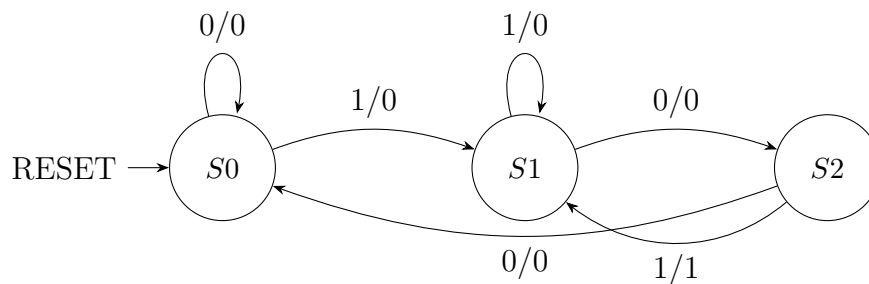
2.1 Overview

A Mealy FSM produces output based on *both* the current state *and* the current input. For the same “101” sequence detector, a Mealy FSM requires only **three states** (compared to four in the Moore version), since the output can be asserted immediately when the final bit of the pattern is seen.

2.2 State Transition Diagram

- **S0**: Initial state / no progress.
- **S1**: Received “1”.
- **S2**: Received “10”.

Transitions are labelled *input/output* (Sb/Z):



2.3 State Table

Current State (Q_1Q_0)	Next State (Sb=0)	Next State (Sb=1)	Output Z
S0 (00)	S0 (00)	S1 (01)	0
S1 (01)	S2 (10)	S1 (01)	0
S2 (10)	S0 (00)	S1 (01)	Z=1 when Sb=1

2.4 State Encoding

State	Q_1Q_0
S0	00
S1	01
S2	10

2.5 Next-State and Output Functions

D_1 (Next Q_1)

$D_1 = 1$ only when the next state is S2 (10), which happens when the current state is S1 and Sb = 0:

$$D_1 = \overline{Q_1} \cdot Q_0 \cdot \overline{Sb}$$

D_0 (**Next** Q_0)

$D_0 = 1$ when the next state is S1 (01), which occurs in:

- S0 when Sb = 1
- S2 when Sb = 1

$$D_0 = \overline{Q_1} \cdot \overline{Q_0} \cdot Sb + Q_1 \cdot \overline{Q_0} \cdot Sb = \overline{Q_0} \cdot Sb$$

Output Z (**Mealy**)

$Z = 1$ only when in state S2 and Sb = 1:

$$Z = Q_1 \cdot \overline{Q_0} \cdot Sb$$

2.6 Comparison with Moore FSM

Property	Moore FSM	Mealy FSM
Number of states	4	3
Output depends on	Current state only	Current state + input
Output timing	1 clock cycle delayed	Immediate (combinational)
Flip-flops required	2	2

3 Exercise 3: Moore FSM – BCD Keypad Lock (Code: 3-4-1)

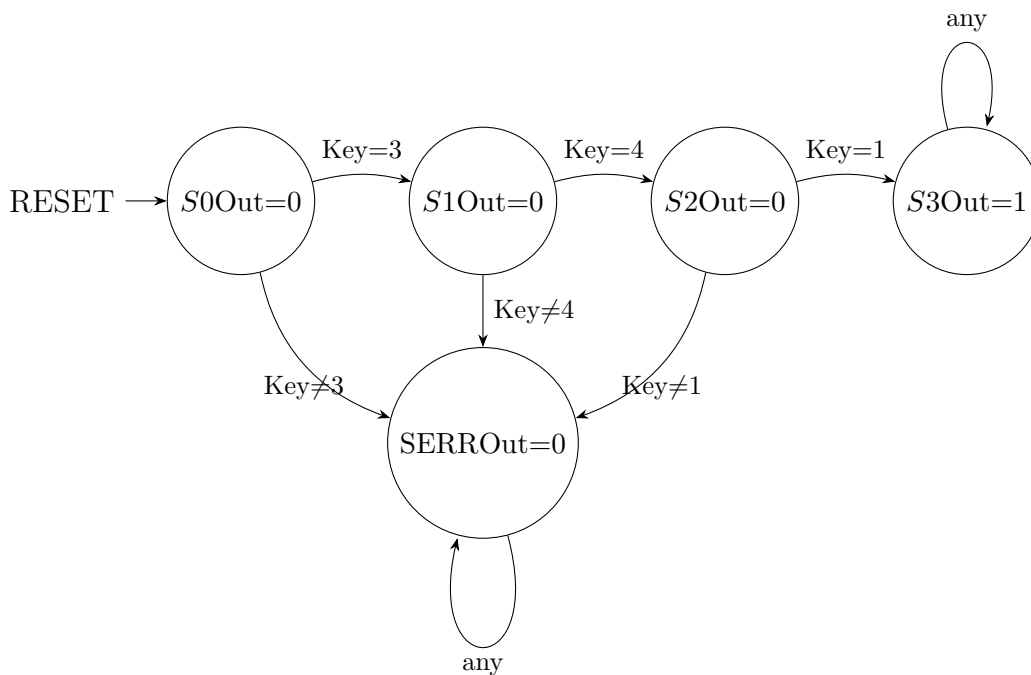
3.1 Overview

A Moore FSM implements a 3-digit BCD keypad lock. The lock opens (output $\text{Out} = 1$) only when the sequence **3, 4, 1** is entered in order on successive clock cycles. The 4-bit BCD input is $\text{Key}[3:0]$, and a synchronous **Reset** returns the FSM to the idle state.

3.2 State Transition Diagram

Five states are needed:

- **S0** ($\text{Out}=0$): Idle / waiting for first digit.
- **S1** ($\text{Out}=0$): Received first correct digit (3 = 0011).
- **S2** ($\text{Out}=0$): Received first two correct digits (3, 4).
- **S3** ($\text{Out}=1$): Received all three correct digits (3, 4, 1) – lock opens.
- **SERR** ($\text{Out}=0$): Error state – an incorrect digit was entered; waits for reset.



3.3 State Table

Current State	Input Key[3:0]	Next State	Output (Out)
S0	0011 (3)	S1	0
S0	any other	SERR	0
S1	0100 (4)	S2	0
S1	any other	SERR	0
S2	0001 (1)	S3	0
S2	any other	SERR	0
S3	any	S3	1
SERR	any	SERR	0

3.4 Output Function

Since this is a Moore FSM:

$$\text{Out} = 1 \iff \text{Current State} = S3$$

3.5 Logisim Implementation Notes

1. The FSM uses **3 D flip-flops** to encode 5 states (minimum 3 bits needed for $\lceil \log_2 5 \rceil = 3$).
2. The 4-bit **Key[3:0]** input is compared with target values using equality comparators.
3. A comparator checks **Key** = 0011 in S0, **Key** = 0100 in S1, and **Key** = 0001 in S2.
4. The output drives an **LED** – lit when **Out** = 1.
5. A synchronous **Reset** forces all flip-flops to 0 (S0) on the next rising clock edge.
6. Input LEDs can be connected to **Key[3:0]** to display the entered digit.

4 Exercise 4: Mealy FSM – BCD Keypad Lock (Code: 2-0-2-5)

4.1 Overview

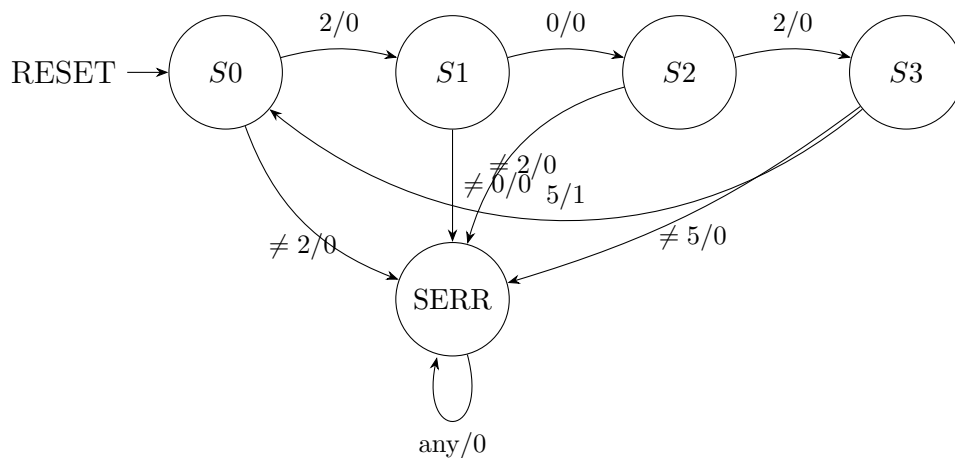
A Mealy FSM implements a 4-digit BCD keypad lock that opens when the sequence **2, 0, 2, 5** is entered. In this Mealy implementation, the output Out is asserted immediately (combinationally) as soon as the fourth correct digit is detected – without waiting for an additional clock cycle.

4.2 State Transition Diagram

Five states track progress through the 4-digit combination:

- **S0**: Idle / waiting for first digit.
- **S1**: Received first correct digit (2 = 0010).
- **S2**: Received “2, 0”.
- **S3**: Received “2, 0, 2”.
- **SERR**: Error – incorrect digit received.

Output Out = 1 is a Mealy output on the transition from S3 when Key = 5:



4.3 State Table

Current State	Input Key[3:0]	Next State	Output (Out)
S0	0010 (2)	S1	0
S0	any other	SERR	0
S1	0000 (0)	S2	0
S1	any other	SERR	0
S2	0010 (2)	S3	0
S2	any other	SERR	0
S3	0101 (5)	S0	1
S3	any other	SERR	0
SERR	any	SERR	0

4.4 Output Function (Mealy)

The output is asserted immediately when in state S3 and Key = 5:

$$\text{Out} = (\text{Current State} = S3) \cdot (\text{Key}[3:0] = 0101)$$

4.5 Comparison with Moore Keypad Lock

Property	Moore (Ex. 3)	Mealy (Ex. 4)
Combination length	3 digits	4 digits
Combination	3, 4, 1	2, 0, 2, 5
Number of states	5	5
Output type	State-dependent only	State + input dependent
Output timing	Registered (1 cycle later)	Combinational (immediate)
Behaviour after unlock	Stays unlocked	Returns to S0

4.6 Logisim Implementation Notes

1. **3 D flip-flops** encode the 5 states (states: 000, 001, 010, 011, 100 for S0–S3 and SERR).
2. Equality comparators check each BCD digit against the target value for each state.
3. The Mealy output `Out` is computed combinatorially: AND of “state = S3” with “Key = 0101”.
4. After a successful unlock, the FSM returns to S0 on the next clock edge, ready for the next entry.
5. A **Reset** input synchronously returns the FSM to S0 at any point.
6. **Input LEDs** on `Key[3:0]` display the currently entered digit; the **output LED** lights immediately when `Out = 1`.